



SPRINT REVIEW - RETROSPECTIVE

Sprint 3

Generazione e gestione di playlist musicali



Gruppo 10

Alfonso Villani - Giovanni Lamberti - Luca Lanzetta - Nicola Liguori



11 → 18 Giugno - Review: 18 Giugno

SP pianificati vs completati

29

Pianificati

29

Completati

0

Rimanenti

100%

dello sprint completato (29/29)

User Story chiuse



US11

Undo

2 SP



US12

Statistiche Home

3 SP



US13

Tag Visivi

3 SP



US14

Auto Playlist

8 SP



US17

Gestione Playlist

3 SP



US18

Visualizzazione Coda

2 SP

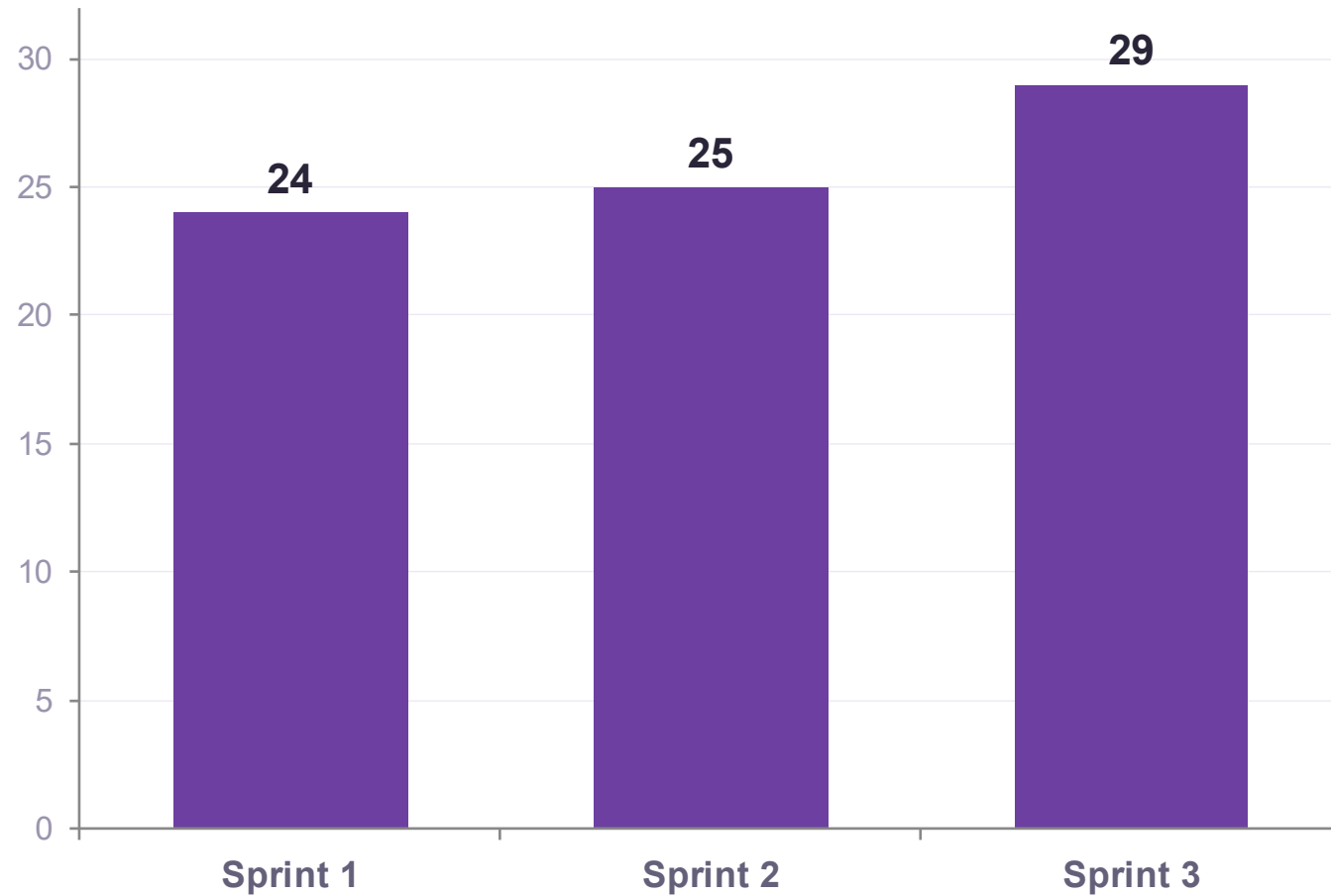


US15

Riordino Live

8 SP

Project Velocity



Velocity media

26 SP/sprint

Calcolo

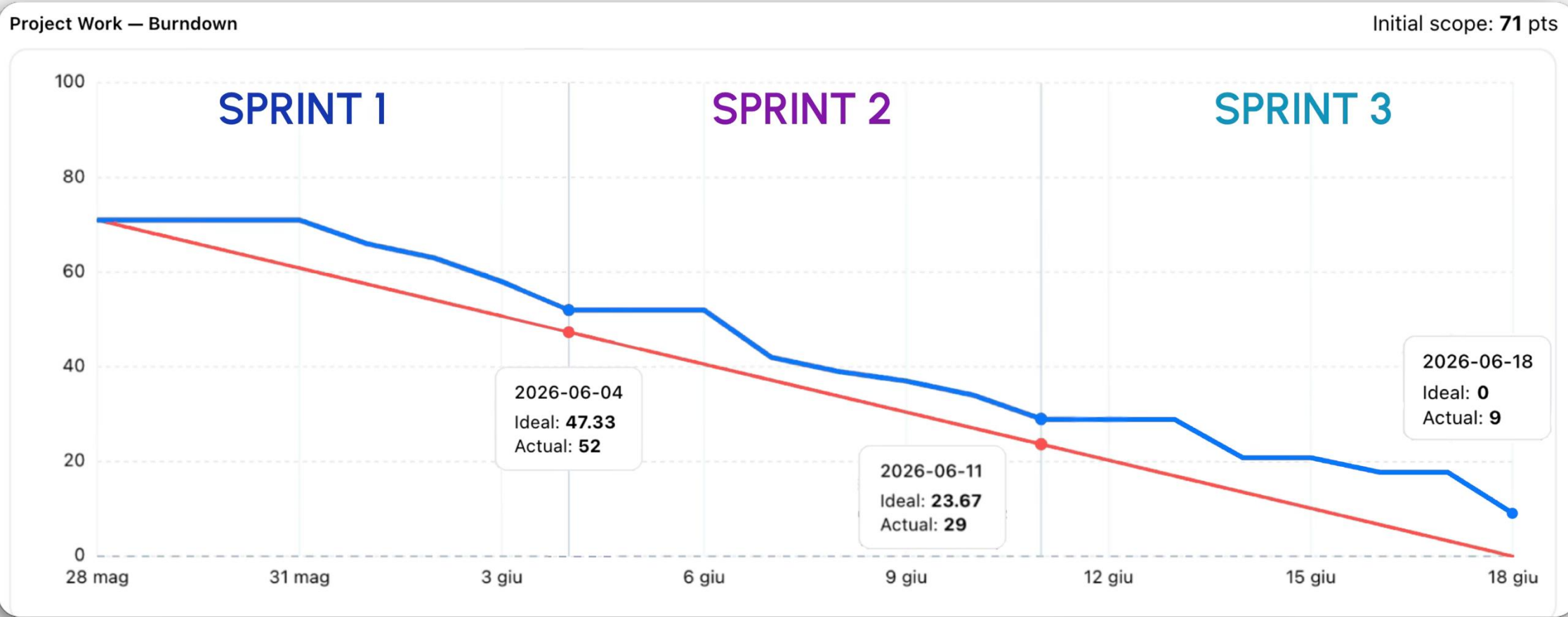
$$(24 + 25 + 29) / 3 = 26$$

Media degli SP completati sui 3 sprint

Crescita costante: +1 SP dallo Sprint 1 al 2, +4 SP dal 2 al 3, con lo Sprint 3 chiuso al 100% dell'impegno pianificato.

Burndown Chart

Andamento degli SP durante tutto il progetto, confrontato con la linea ideale di burndown.



Review: problemi riscontrati e soluzioni



US14 - Auto Playlist

Validazione di molti parametri di input

Gestire e validare insieme genere, anno e tag visivi rischiava di generare un blocco di codice rigido, con una lunga catena di if annidati.

Soluzione - Pattern Strategy

La logica di filtraggio è stata disaccoppiata: ogni criterio di selezione vive in una propria strategia, associabile alla playlist e aggiornabile dinamicamente.



US15 - Riordino Live

Drag & Drop e disallineamento UI / Model

Spostare una `TrackUIComponentItem` in una lista dinamica rischiava di disallineare l'indice mostrato nella UI dalla posizione reale nel Model, con blocchi grafici o `IndexOutOfBoundsException`.

Soluzione - JavaFX ClipboardContent

Salvataggio della posizione di partenza all'inizio del trascinamento tramite le API native di JavaFX, e riposizionamento esatto al rilascio del cursore.

Sprint Review



Granularità delle task

Rispetto ai due sprint precedenti le task sono risultate ben dimensionate e lo sviluppo è proseguito in modo lineare, anche grazie alla solida base costruita prima e a refactoring tempestivi.

Miglioramento



Distribuzione del lavoro

Il carico non è stato del tutto uniforme: le US ad alta complessità (US14, US15) hanno concentrato l'effort su parte del team. Lo sprint si è chiuso comunque, ma serve un bilanciamento migliore già in fase di planning.

Da migliorare

Sprint Retrospective

Starfish Diagram



Start Doing

- Mockup grafici prima di sviluppare le view
- Test automatizzati in parallelo alla logica



More Of

- Sessioni di pair programming sul design
- Review interne preventive prima del merge



Keep Doing

- Ottima comunicazione interna al team
- Design Pattern per isolare la logica complessa



Less Of

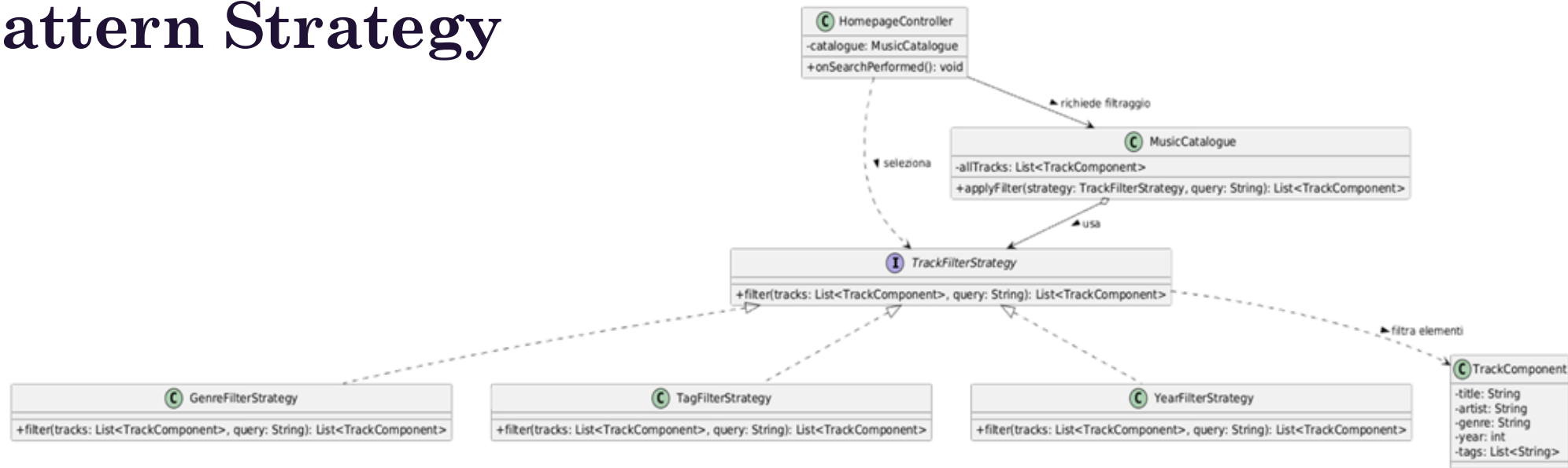
- Stime ottimistiche dell'effort
- Test manuali ripetitivi sulla UI JavaFX



Stop Doing

- Rimandare la rifinitura grafica a fine sprint
- Portare avanti troppe US in parallelo

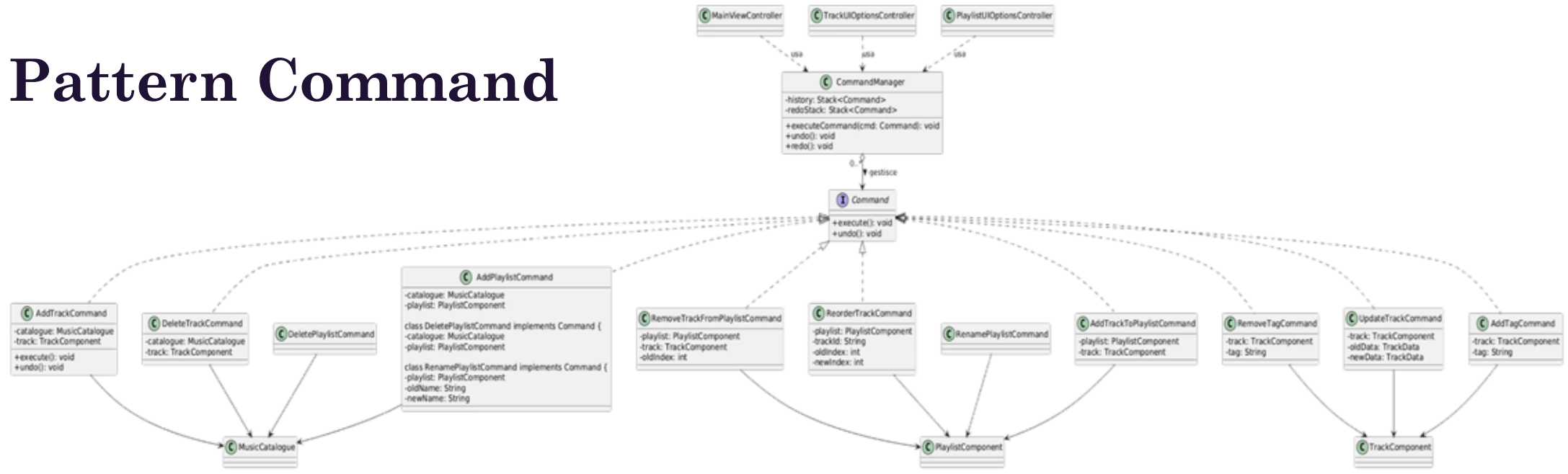
Pattern Strategy



Come l'abbiamo applicato (US14)

- Ogni criterio di filtraggio (genere, anno, tag visivi) è isolato in una propria strategia.
- Le strategie sono intercambiabili a runtime e si associano dinamicamente alla playlist.
- Niente più catene di if annidati: il codice resta aperto a nuovi criteri senza riscrittura.
- Logica di generazione testabile in isolamento, una strategia alla volta.

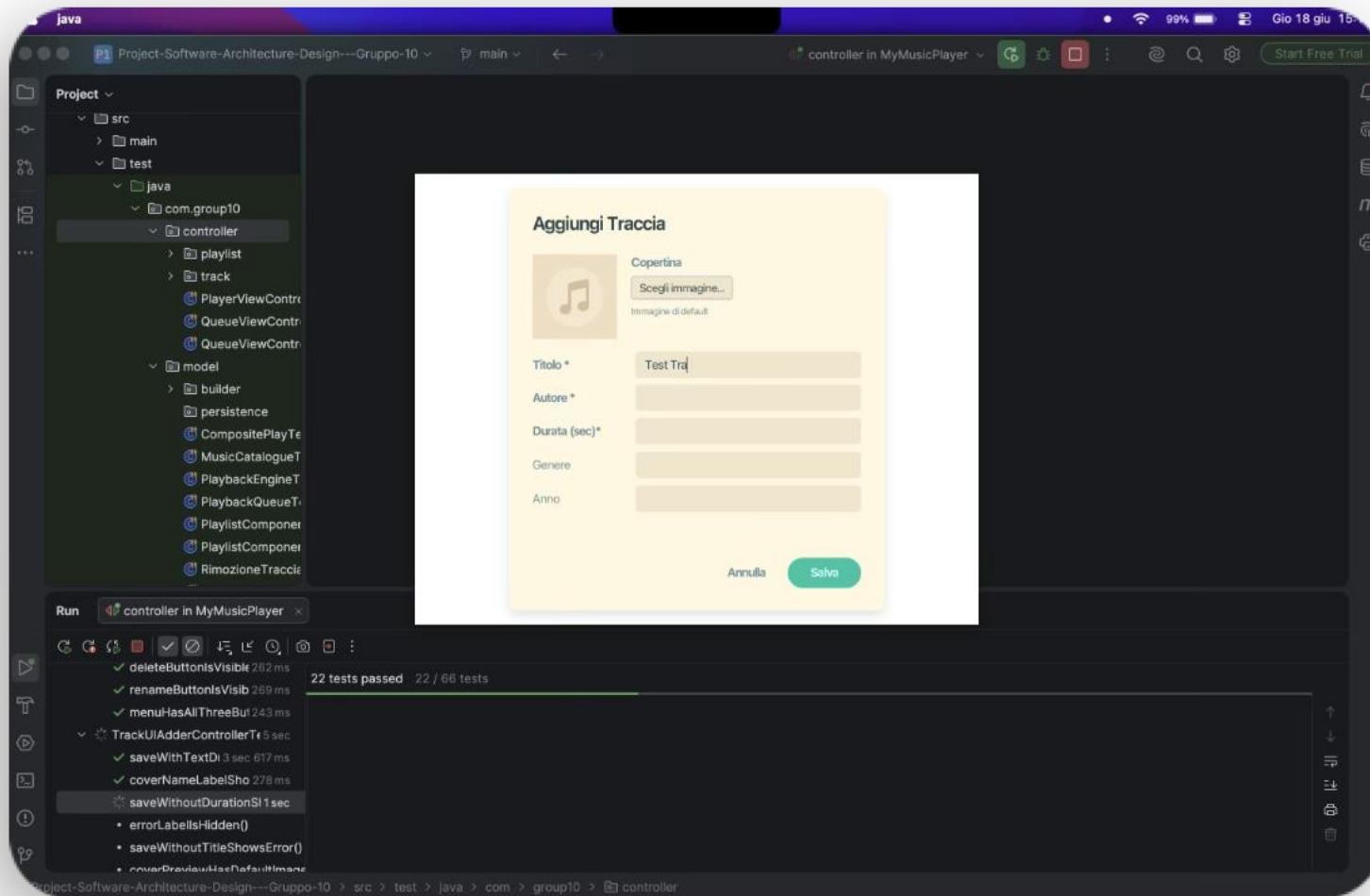
Pattern Command



Come l'abbiamo applicato (US11)

- Ogni azione dell'utente (aggiunta, rimozione, riordino di una traccia) è incapsulata in un oggetto **Command** con il proprio metodo di esecuzione e di annullamento.
- I comandi eseguiti finiscono in uno storico: l'undo ripristina lo stato precedente richiamando l'operazione inversa dell'ultimo comando.
- La UI non conosce i dettagli delle operazioni, si limita a creare e lanciare comandi. Questo tiene separato chi richiede l'azione da chi la esegue.
- Aggiungere una nuova operazione annullabile vuol dire scrivere una nuova classe **Command**, senza toccare il codice già esistente.

Test FX implementati



Cosa abbiamo coperto

- Test automatizzati sul Model con JUnit, scritti accanto alla logica di business.
- TestFX sulla UI JavaFX, in particolare drag & drop e riordino live (US15).
- Verifica dei criteri di generazione della Auto Playlist (US14).
- Obiettivo dichiarato in retrospective: ridurre i test manuali ripetitivi sull'interfaccia.

Schermata di Release

Stato dell'applicazione al termine dello Sprint 3: auto playlist, riordino live e visualizzazione della coda.

